



Political Patricians (patricians)

Un censeur romain surveille N patriciens aux allégeances politiques douteuses. Une *faction* est un groupe de un ou plusieurs patriciens. Chaque patricien appartient à **exactement une** faction. Le censeur souhaite déterminer à quelle faction appartient chaque patricien.

Pour ce faire, le censeur organisera à plusieurs reprises des banquets auxquels les N patriciens seront conviés. Lors de chaque banquet, les patriciens peuvent être répartis à différentes tables, et le censeur contrôle cette répartition. Un patricien est *déconcerté* s'il n'y a aucun autre patricien de la même faction assis à la même table.

En tant que maître de cérémonie, le censeur saura après chaque banquet quels patriciens étaient déconcertés. Votre tâche est d'aider le censeur à déterminer les factions des patriciens sans organiser trop de banquets.

Implémentation

Vous devez soumettre un seul fichier source `.cpp`.



Parmi les pièces jointes du problème, vous trouverez un squelette `patricians.cpp` avec une implémentation d'exemple.

Vous devez implémenter la fonction suivante :

C++	<code>vector<int> find_factions(int N)</code>
-----	---

- L'entier N représente le nombre de patriciens. Les patriciens sont numérotés de 0 à $N - 1$.
- La fonction doit retourner un tableau P de N entiers représentant les factions des patriciens. $P[i]$ doit être égal à $P[j]$ si et seulement si les patriciens i et j appartiennent à la même faction. Il n'y a pas d'autre contrainte sur les valeurs des entiers de P .

Votre programme peut appeler la fonction suivante, qui est déjà définie dans le grader :

C++	<code>vector<bool> organize_banquet(const vector<int>& Q)</code>
-----	--

- Le tableau Q de N entiers représente la répartition des places : le patricien i sera assis à la table $Q[i]$. Il n'y a aucune contrainte sur les valeurs des entiers de Q : les tables peuvent porter n'importe quel numéro, et la seule information pertinente est que si $Q[i] = Q[j]$, alors les patriciens i et j sont assis à la même table.
- La fonction retournera un tableau A de N booléens. $A[i]$ sera vrai si et seulement si le patricien i est déconcerté lors du banquet.

Le grader est **non adaptatif** et chaque soumission est évaluée sur le même ensemble de tests. C'est-à-dire que les factions de chaque patricien sont fixées à l'avance pour chaque test, et ne dépendent pas des appels à `organize_banquet` effectués par votre programme.

Évaluateur d'exemple

Parmi les fichiers joints à cet énoncé, vous trouverez une version simplifiée du grader utilisé lors de l'évaluation. Vous pouvez l'utiliser pour tester vos solutions en local. Ce grader lit les données depuis `stdin`, appelle la fonction `find_factions` et écrit le résultat dans `stdout` selon le format suivant.

Le fichier d'entrée est composé de 2 lignes, contenant :

- Ligne 1 : l'entier N .
- Ligne 2 : N entiers décrivant la faction de chaque patricien.

La sortie contient une seule ligne, indiquant le score obtenu par votre solution sur ce test. Si le score est 0, un message d'erreur bref est affiché sur `stderr` pour en indiquer la cause (factions incorrectes, requête invalide ou trop de requêtes).

Contraintes

- $4 \leq N \leq 2000$.

Vous pouvez appeler `organize_banquet` au maximum 30000 fois.

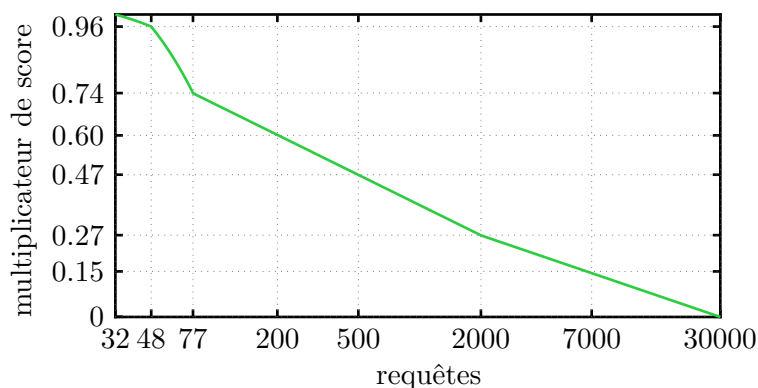
Score

Votre programme sera testé sur plusieurs tests, regroupés en sous-tâches. Le multiplicateur de score associé à une sous-tâche sera le minimum des multiplicateurs de score obtenus sur chacun des tests de cette sous-tâche.

- **Sous-tâche 0 [0 points]**: Exemples.
- **Sous-tâche 1 [19 points]**: Il y a exactement deux factions.
- **Sous-tâche 2 [29 points]**: Chaque faction contient exactement deux patriciens.
- **Sous-tâche 3 [52 points]**: Pas de contraintes additionnelles.

Le multiplicateur de score obtenu sur un test est déterminé comme suit.

Nombre de requêtes	Multiplicateur de score
≤ 32	1.0
$33 \leq x < 48$	$1.08 - 0.0025x$
48	0.96
$48 < x < 77$	$1.32413 - 0.00758x$
77	0.74
$77 < x < 2000$	$1.366 - 0.1 \log_2(x)$
2000	0.27
$2000 < x \leq 30000$	$1.027 - 0.0691 \log_2(x)$



Le score obtenu sur une sous-tâche est le minimum des multiplicateurs de score obtenus sur les tests de la sous-tâche, multiplié par le score de base de la sous-tâche.

Exemples

Évaluateur	Solution
find_factions(5)	
	organize_banquet([0,0,1,1,2])
return [1,1,1,1,1]	
	organize_banquet([0,0,0,1,1])
return [0,1,0,1,1]	
	organize_banquet([0,1,2,1,1])
return [1,0,1,0,1]	
	return [0,1,0,1,2]

Évaluateur	Solution
find_factions(4)	
	organize_banquet([1,1,0,0])
return [1,1,1,1]	
	organize_banquet([1,1,1,2])
return [0,1,0,1]	
	organize_banquet([0,1,1,1])
return [1,0,1,0]	
	return [0,1,0,1]

Explication

Dans le premier exemple, les patriciens 0 et 2 sont dans une faction, les patriciens 1 et 3 sont dans une autre, et le patricien 4 est dans une faction à lui seul. Lors du premier banquet, tous les patriciens sont déconcertés. Lors du second banquet, les patriciens 0 et 2 ne sont pas déconcertés car ils sont assis ensemble. Lors du troisième banquet, les patriciens 1 et 3 ne sont pas déconcertés car ils sont assis ensemble.